FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Crisostomo, Jerome M.
Section:	AN42
Professor:	Ma'am. Crisola G. Tan
Date:	10/04/2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice

- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule

3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
   Definitely RBR since you only have to set up an if-then rule in lists, dictionaries, or sets, then check which rule match.
2. Which is more flexible in solving unstructured problems?
   CBR is generally more flexible for unstructure problems because it can draw on similar past situations even when the problem doesn't fit neatly into predefined rules, then adapt a previous solution.
3. Which is better for situations with continuously changing data?
   Case-based reasoning is often better when data or situations keep changing, because you can add new cases as experience accumulates and use the most similar ones.
4. Examples of real-world applications for both.

Rule-based reasoning: A troubleshooting system that follows rules such as, “If the printer is offline and the cable is disconnected, reconnect the cable.”
Case-based reasoning: A doctor or mechanic compares a new problem with similar past cases and adapts a previous diagnosis or repair.

Your Code - Laboratory Tasks ⇩

```
# ================================================================
# PART 1: Knowledge Representation (KR)
# Task 1A: Knowledge Base (KB)
# ================================================================
facts = {
    "temperature": 34,
    "humidity": 82,
    "is_raining": True,
    "motion_detected": True,
    "air_quality_index": 145,
    "ac_status": "standby",
    "window_open": False
}

# ================================================================
# PART 2: Implement Rule-Based Reasoning (RBR)
# Task 2A: IF-THEN Rules
# ================================================================
def rule_hot(facts):
    if facts.get("temperature", 0) > 30:
        return "Turn on the aircon"
    return None

def rule_rain(facts):
    if facts.get("is_raining"):
        return "Bring umbrella"
    return None

def rule_humidity(facts):
    if facts.get("humidity", 0) > 70:
        return "Activate dehumidifier mode"
```

```
    return None

def rule_air_quality(facts):
                if    facts.get("air_quality_index",    0)    >    100    and
facts.get("motion_detected"):
        return "Turn on air purifier"
    return None


# ================================================================
# Task 2B: Implement an Inference Engine
# ================================================================
def inference_engine(knowledge_base, rule_set):
    """
    Inference Engine:
    1. Reads the knowledge base facts.
    2. Iterates through all available rules.
    3. Evaluates condition and collects all triggered actions.
    """
    inferred_actions = []

    for rule in rule_set:
        action = rule(knowledge_base)
        if action is not None:
            inferred_actions.append(action)

    return inferred_actions


# ================================================================
# Task 2C: Display Output
# ================================================================
rule_set = [rule_hot, rule_rain, rule_humidity, rule_air_quality]

# Run the inference engine
actions = inference_engine(facts, rule_set)
```

```
# Display formatted output
print("Rule-Based Reasoning Actions:")
if actions:
    for action in actions:
        print(f"- {action}")
else:
    print("- No actions required.")

# ============================================================
# PART 3: Case-Based Reasoning (CBR)
# Task 3A: Build a Case Base
# ============================================================
case_base = [
    {
        "id": 1,
           "problem": {"temperature": 32, "is_raining": True, "humidity":
85},
        "solution": "Turn on AC to Cool mode, enable dehumidification, and
stay indoors."
    },
    {
        "id": 2,
           "problem": {"temperature": 18, "is_raining": True, "humidity":
75},
        "solution": "Wear a raincoat, keep windows closed, and turn on the
room heater."
    },
    {
        "id": 3,
          "problem": {"temperature": 35, "is_raining": False, "humidity":
40},
        "solution": "Turn on the AC at full power, stay hydrated, and wear
light clothing."
    },
    {
        "id": 4,
```

```
        "problem": {"temperature": 24, "is_raining": False, "humidity":
50},
            "solution": "Open the windows for fresh air and turn on a
low-speed ceiling fan."
    }
]



# ============================================================
# Helper / Similarity Function (Retrieve Step)
# ============================================================
def calculate_similarity(new_problem, case_problem):
    """
      Computes a normalized similarity score (0.0 to 1.0) between two
problems:
    - Temperature diff normalized over a 40-degree range
    - Humidity diff normalized over a 100-percent range
    - Boolean match on is_raining
    """
            temp_sim   =   1.0   -   (abs(new_problem["temperature"]   -
case_problem["temperature"]) / 40.0)
             humid_sim   =   1.0   -   (abs(new_problem["humidity"]   -
case_problem["humidity"]) / 100.0)
              rain_sim   =   1.0   if   new_problem["is_raining"]   ==
case_problem["is_raining"] else 0.0

    # Total weighted average similarity
    total_score = (temp_sim * 0.4) + (humid_sim * 0.3) + (rain_sim * 0.3)
    return max(0.0, total_score)

def retrieve_most_similar_case(new_problem, cases):
    """Finds and returns the case with the highest similarity score."""
    best_case = None
    highest_score = -1.0

    for case in cases:
        score = calculate_similarity(new_problem, case["problem"])
```

```
        if score > highest_score:
            highest_score = score
            best_case = case

    return best_case, highest_score


# ================================================================
# Task 3C: Reuse Step
# ================================================================
def reuse_solution(best_case):
    """Returns the solution from the most similar past case."""
    print(f"\n[Reuse] Retrieved Solution from Case #{best_case['id']}:")
    print(f'"{best_case["solution"]}"')
    return best_case["solution"]


# ================================================================
# Task 3D: Revise Step
# ================================================================
def revise_solution(suggested_solution):
      """Allows manual adjustment of the proposed solution or keeps it
unchanged."""
    print("\n--- Revise Step ---")
      user_choice = input("Do you want to adjust this solution? (y/n):
").strip().lower()

    if user_choice == 'y':
            revised_solution = input("Enter revised/adjusted solution:
").strip()
        print("[Revise] Solution updated manually.")
        return revised_solution
    else:
        print("[Revise] Solution accepted without changes.")
        return suggested_solution
```

```
# ================================================================
# Task 3E: Retain Step
# ================================================================
def retain_case(cases, new_problem, final_solution):
    """Packages the new problem and confirmed solution, adding it to the
case base."""
    new_entry = {
        "id": len(cases) + 1,
        "problem": new_problem,
        "solution": final_solution
    }
    cases.append(new_entry)
    print(f"\n[Retain] Successfully stored new Case #{new_entry['id']} in
Case Base!")


# ================================================================
# Demonstration / Execution
# ================================================================
if __name__ == "__main__":
    # A new incoming query/problem
    current_problem = {
        "temperature": 33,
        "is_raining": True,
        "humidity": 82
    }

    print("Current Query Problem:", current_problem)

    # 1. RETRIEVE
    best_match, similarity = retrieve_most_similar_case(current_problem,
case_base)
    print(f"[Retrieve] Best Match: Case #{best_match['id']} (Similarity:
{similarity * 100:.1f}%)")

    # 2. REUSE (Task 3C)
    proposed_solution = reuse_solution(best_match)
```

```
    # 3. REVISE (Task 3D)
    final_solution = revise_solution(proposed_solution)

    # 4. RETAIN (Task 3E)
    retain_case(case_base, current_problem, final_solution)

    # Verify total cases in knowledge base
    print(f"\nTotal Cases now stored in Case Base: {len(case_base)}")
```